

AI-Based Smart Vehicle Insurance Claim Assessment

Complete Technical Architecture & Implementation Plan

FastAPI + ONNX Runtime (YOLOv8s-seg) + PostgreSQL + OpenCV / pHash + React TS Portal

STATUS: APPROVED ARCHITECTURE

ESCALATION: RULES E1-E9

DETERMINISTIC PRICING MATRIX

1. Project Overview & Scope Alignment

This project designs and implements an AI-assisted claim assessment platform that automates the triage stage of vehicle insurance claims. Policyholders submit photographs of exterior cosmetic damage, the system validates photo integrity, localizes damaged vehicle parts, classifies damage type and severity using computer vision, estimates repair costs against a structured relational rate matrix, and routes each claim through a deterministic decision engine into one of two paths: AUTO_APPROVED for clean, low-value cosmetic claims, or SURVEYOR_REVIEW for anything structurally sensitive, high-value, low-confidence, or flagged for anomalies.

The system deliberately scopes AI to triage and cost pre-filling, strictly preserving human authority over any claim carrying risk of error, overpayment, underpayment, or fraud.

[IMPORTANT] Dual Database & Object Storage Dev/Production Architecture

PostgreSQL is the designated production relational database. To ensure zero-friction developer bootstrap without requiring Docker or a running PostgreSQL daemon, SQLAlchemy ORM supports both PostgreSQL and SQLite via DATABASE_URL. S3/MinIO storage client supports both AWS/MinIO endpoints and a transparent local filesystem fallback.

[NOTE] Model Inference & ONNX Serving Standards

Per specification, ONNX Runtime (opset 17) executes Model 1 (9 Vehicle Parts) and Model 2 (6 Damage Types) independently on 640x640 letterboxed inputs. A synthetic ONNX generator guarantees deterministic real-tensor inference during tests and offline demonstrations.

2. System Architecture & Directory Structure

```
c:/Major project/
|-- antigravity_execution_plan.json      # Dynamic task execution tracker
|-- backend/
|   |-- app/
|   |   |-- api/ (auth.py, claims.py, rates.py, router.py)
|   |   |-- core/ (config.py, security.py)
|   |   |-- db/ (database.py, models.py, seed.py)
|   |   |-- schemas/ (claim.py, rates.py)
|   |   |-- services/ (cv_inference.py, mask_fusion.py, cost_engine.py,
|   |   |               decision_engine.py, photo_validator.py, storage.py)
|   |   |-- main.py                      # FastAPI application entrypoint
|   |-- tests/ (test_mask_fusion.py, test_cost_engine.py, test_decision_engine.py,
|   |          test_photo_validation.py, test_claims_api.py)
|-- frontend/ (React + TypeScript + Vite + Tailwind CSS Surveyor Review Portal)
|-- demo_runner.py                      # Automated end-to-end verification scenario
```

3. Technical Modules Specification

Module 1: Computer Vision & Mask Fusion Pipeline

Two independent models are deployed to decouple part boundaries from damage contours:

- Model 1 (Vehicle Parts): 9 classes (bumper_front, bumper_rear, door, fender, headlamp, taillamp, mirror, hood, windshield).
- Model 2 (Damage Types): 6 classes (scratch, dent, crack, shatter, paint_chip, misalignment).

The mask fusion algorithm discards noise blobs and exterior detections, then attributes damage to the best-overlapping part by calculating $\text{intersection_area} / \text{part_area}$.

```
DAMAGE_TYPE_WEIGHT = {'scratch': 1, 'paint_chip': 1, 'dent': 2, 'misalignment': 3, 'crack': 3, 'shatter': 4}

def grade_severity(area_ratio: float, damage_type: str) -> str:
    score = area_ratio * DAMAGE_TYPE_WEIGHT.get(damage_type, 2)
    if score < 0.15: return 'MINOR'
    elif score < 0.45: return 'MODERATE'
    else: return 'SEVERE'
```

Damage that cannot be attributed to any part (ratio < 0.05) is flagged as UNATTRIBUTED (triggers E1).

Module 2: Deterministic Cost Estimation Engine

Cost estimation is purely deterministic, looking up relational rate_matrix rows for (tier_id, part_id, damage_type, severity_band). No LLM or heuristic regression is permitted for financial calculations.

```
decision = REPLACE if (severity_band == 'SEVERE' or damage_type in {'shatter', 'crack'}
                      or repair_cost_base == 0) else REPAIR
base_cost = replace_cost_base if decision == REPLACE else repair_cost_base
labor_hrs = replace_labor_hrs if decision == REPLACE else repair_labor_hrs
labor_cost = labor_hrs * tier.labor_rate_hr
line_total = base_cost + labor_cost
subtotal = sum(line_total across all valid line items)
payable = max(subtotal - policy.deductible, Decimal('0.00'))
```

Module 3: Deterministic Decision Engine (Escalation Rules E1 - E9)

A claim is evaluated against nine strict, explainable escalation rules. A single failure routes the claim to mandatory SURVEYOR_REVIEW. A claim is AUTO_APPROVED only when zero escalation rules fire, all damages are in cosmetic whitelist (scratch, dent, paint_chip), and all severity bands are MINOR or MODERATE.

Rule	Trigger Condition	System Routing Rationale
E1	Unattributed damage region present	Damage not covered by any part model mask
E2	Damaged part flagged structural	Frame/mechanical adjacency (hood, fender rails)
E3	Part or damage confidence < 0.80	High uncertainty in CV segmentation inference
E4	More than 2 distinct parts damaged	Multi-panel collision exceeding cosmetic triage
E5	Any line item graded SEVERE	High severity structural/replacement threshold
E6	Total payable > Rs. 25,000 ceiling	Exceeds maximum automated payout threshold
E7	Fraud heuristic score > 0	Perceptual duplicate hash or severe EXIF mismatch
E8	Missing rate-matrix row for line item	Unpriced claim cannot be silently zero-costed
E9	Photo validation failed	Blurry image (Laplacian var < threshold) / no car

Module 4: Photo Validation & Fraud Heuristics

- Blur Detection: Laplacian variance of grayscale image. Threshold < 100.0 rejects photo immediately.
- Vehicle Presence Check: Detects automotive contour / vehicle presence covering > 15% of frame.
- Duplicate Photo Detection: Computes 64-bit perceptual hash (imagehash.phash) and calculates Hamming distance against past claims in photo_hash_store. Distance <= 5 against a different policyholder flags fraud (E7).
- EXIF Consistency Check: Compares EXIF capture datetime against reported incident datetime. Missing GPS is soft signal.

Module 5: Backend Gateway & Human-in-the-Loop Override Audit

FastAPI handles authentication (JWT, role-separated: policyholder / surveyor), claim submission, and background processing. Claims progress through statuses: SUBMITTED -> PROCESSING -> ASSESSED -> SURVEYOR_REVIEWED / APPROVED / REJECTED. When a surveyor adjusts pricing or overrides decisions, every change is written to claim_overrides preserving previous and new values, surveyor_id, timestamp, and justification for total regulatory compliance.

Module 6: React + TypeScript Surveyor Portal

A component-based UI designed specifically for claims adjusters:

- Claims Triage Queue: Filterable by status, escalation reasons, and date.
- Interactive Annotation Canvas: Displays original photos with toggleable mask overlays for vehicle parts and damage contours.
- Editable Cost Table: Real-time repair/replace toggle, labor hour editing, and live subtotal/deductible recalculation.
- Decision Audit Card: Visual indicators for each E1-E9 rule trigger.
- Policyholder Submission Flow: Simple upload form for policyholders with real-time feedback.

4. Rigorous Verification & Testing Strategy

The system includes five dedicated automated test suites:

1. test_mask_fusion.py: Verifies intersection_area / part_area and severity bands across synthetic masks.
2. test_cost_engine.py: Verifies repair-vs-replace formulas, labor multipliers, deductible calculations, and missing row exceptions.
3. test_decision_engine.py: Comprehensive test for each escalation condition E1 through E9 plus a clean AUTO_APPROVED scenario.
4. test_photo_validation.py: Tests Laplacian blur filter, duplicate pHash Hamming distance, and EXIF plausibility.
5. test_claims_api.py: End-to-end integration test verifying claim submission, async background triage, override logging, and decision finalizing.